UNIWERSYTET WSB MERITO WARSZAWA

Kryptologia i kryptografia

dr inż. Piotr Bobiński
piotr.bobinski@warszawa.merito.pl

Studia niestacjonarne Ćwiczenia 1 z 4

Plan

- Powitanie
- Regulacje
- O przedmiocie i plan wykładów

Regulacje formalne

§ 21

1. Zasady uczestniczenia w zajęciach (w tym zaliczenia) są określone w Karcie przedmiotu.

2. Karta przedmiotu jest udostępniana studentom w systemie internetowym Uczelni nie później, niż dwa tygodnie po rozpoczęciu zajęć z danego przedmiotu.

§ 19

2. Wykłady w Uczelni nie są obowiązkowe.

3. Udział w ćwiczeniach i konwersatoriach jest obowiązkowy. Wykładowca określa na początku zajęć dopuszczalną liczbę nieobecności.

Warunki zaliczenia

- **Ćwiczenia**
  - Przygotowanie projektu w dowolnej technologii 70
  - Przygotowanie dokumentacji dla wykonywanego projektu 30
  - Suma punktów 100

Liczba punktów	Ocena
91-100%	5
81-90%	4,5
71-80%	4
61-70%	3,5
51-60%	3
poniżej 50%	2

Sprawy techniczne

- **8 godzin (4 spotkania)**

- Moodle – materiały i komunikacja grupowa
- Email – komunikacja indywidualna
  - piotr.bobinski@warszawa.merito.pl
  - Wiadomości **TYLKO** z adresu uczelnianego

Literatura (przykłady)

https://www.merito.pl/warszawa/strefa-studenta/biblioteka
https://ebookpoint.pl/
https://biblio.ebookpoint.pl/biblioteka

- Podawane na bieżąco

Plan ćwiczeń

1. **Podstawy i kryptografia symetryczna**
2. Funkcje skrótu i struktury danych
3. Kryptografia asymetryczna
4. Protokoły i zastosowania zaawansowane
    1. Blockchain

Tematyka projektów

- Zamieszczona na Moodl’u (będzie)

Repozytorium

- https://github.com/pbobinski-WSB/WSB_KIK_2026L

Fundamenty i kryptografia symetryczna

- Cel:
  - Zrozumienie, czym jest losowość, poufność i dlaczego "własne szyfry" są złe.
- Wstęp, entropia i losowość
  - Teoria:
    - Zasada kerckhoffsa, bezpieczeństwo obliczeniowe vs absolutne (one-time pad), entropia shannona, generatory liczb pseudolosowych (PRNG vs CSPRNG).
  - Zadania:
    - Analiza statystyczna (testy losowości) wyjścia rand() vs /dev/urandom.
    - Implementacja szyfru vernama (otp) i pokazanie problemu "many-time pad" (atak na powtórzony klucz).
- Szyfry strumieniowe i rejestry przesuwne
  - Teoria:
    - Konstrukcja LFSR (linear feedback shift register), pojęcie okresu, nowoczesne szyfry strumieniowe (chacha20).
  - Zadania:
    - Implementacja LFSR i atak berlekampa-masseya (odtworzenie wielomianu na podstawie wyjścia).
    - Szyfrowanie pliku przy użyciu chacha20 (np. W pythonie/C).

Fundamenty i kryptografia symetryczna

- Szyfry blokowe i tryby operacyjne (to jest kluczowe!)
  - Teoria:
    - Budowa aes (spn) vs des (feistel). Tryby operacyjne: ECB, CBC, CTR. Dlaczego ECB to zło (przykład z pingwinem tux).
  - Zadania:
    - Zaszyfrowanie obrazka bitmapowego w trybie ECB i CBC – wizualne porównanie.
    - Atak bit-flipping w trybie ctr (zmiana treści zaszyfrowanej wiadomości bez znajomości klucza).
- Uwierzytelnianie i aead
  - Teoria:
    - MAC (message authentication code), HMAC. Problem chosen ciphertext attack. Szyfrowanie uwierzytelnione (GCM, poly1305).
  - Zadania:
    - Atak padding oracle na tryb CBC.
    - Implementacja bezpiecznego kanału z wykorzystaniem aes-gcm.

Funkcje skrótu i struktury danych

- Cel:
  - Przygotowanie gruntu pod blockchain.
- Funkcje skrótu (hash)
  - Teoria:
    - Własności: odporność na kolizje, odporność na przeciwobraz. Rodzina SHA-2 vs SHA-3 (keccak).
    - Paradoks urodzinowy.
  - Zadania:
    - Szukanie kolizji na "skróconym" hashu (np. Pierwsze 4 bajty).
    - Implementacja ataku length extension attack na naiwną konstrukcję h(klucz || wiadomość) (ważne, by zrozumieć po co jest hmac).
- Drzewa merkle i struktury uwierzytelnione
  - Teoria:
    - Drzewa skrótów, dowód przynależności (merkle proof). Wykorzystanie w systemach plików (IPFS) i blockchainie.
  - Zadania:
    - Implementacja drzewa merkle od zera.
    - Generowanie i weryfikacja ścieżki dowodowej (merkle path) dla konkretnej transakcji/pliku.

Kryptografia asymetryczna

- Cel:
  - Matematyka stojąca za tożsamością w sieci.
- Teoria liczb – niezbędnik
  - Teoria:
    - Grupy, ciała skończone, arytmetyka modularna, małe twierdzenie fermata, algorytm euklidesa, chińskie twierdzenie o resztach (CRT).
  - Zadania:
    - Implementacja szybkiego potęgowania modularnego.
    - Obliczanie odwrotności modulo (rozszerzony euklides). Warto użyć pythona lub sagemath.
- Rsa i bezpieczeństwo klucza publicznego
  - Teoria:
    - Algorytm RSA. Problemy faktoryzacji. Dlaczego "textbook RSA" jest niebezpieczny (brak paddingu OAEP).
  - Zadania:
    - Atak na RSA z małym wykładnikiem publicznym (e=3).
    - Atak cykliczny lub atak wiedemanna na faktoryzację (dla małych liczb).

Kryptografia asymetryczna

- Protokół Diffiego-Hellmana i logarytm dyskretny
  - Teoria:
    - Problem logarytmu dyskretnego (DLP). Wymiana kluczy DH. Atak man-in-the-middle.
  - Zadania:
    - Implementacja DH na małych liczbach.
    - Algorytm "baby-step giant-step" do łamania logarytmu dyskretnego.
- Krzywe eliptyczne (ecc) – podstawa bitcoina i nie tylko
  - Teoria:
    - Krzywe nad ciałami skończonymi. Dodawanie punktów. Krzywa secp256k1 (używana w bitcoinie).
    - Dlaczego ECC jest lepsze od RSA?
  - Zadania:
    - Dodawanie punktów na krzywej "na piechotę" (w kodzie).
    - Generowanie pary kluczy (prywatny/publiczny) dla krzywej secp256k1.

Protokoły i zastosowania zaawansowane

- Cel:
  - Wyjście poza proste szyfrowanie.
- Podpisy cyfrowe
  - Teoria:
    - RSA-PSS vs ECDSA vs eddsa. Podpisy Schnorra (ważne dla bitcoina/taproot).
  - Zadania:
    - Implementacja ECDSA (z uwzględnieniem niebezpieczeństwa użycia tego samego k – nonce reuse attack, który pozwolił wykraść klucze z PS3).
- Dowody z wiedzą zerową (zero knowledge proofs) i secret sharing
  - Teoria:
    - Protokół schnorra (identyfikacja). Schemat dzielenia sekretu Shamira. Wstęp do zk-snarks (teoretycznie).
  - Zadania:
    - Implementacja dzielenia sekretu Shamira (np. Dzielimy klucz prywatny na 5 fragmentów, 3 potrzebne do odtworzenia).

Finał – Blockchain

- Cel:
  - Złożenie wszystkiego w całość.
- Blockchain – Struktura danych i "Mining"
  - Teoria:
    - Blockchain jako łańcuch bloków hashowanych. Proof of Work (Hashcash). Trudność kopania.
  - Zadania:
    - Symulacja kopania bloku (szukanie nonce, dla którego hash ma N zer wiodących).
    - Analiza nagłówka bloku Bitcoina (wersja, prev_hash, merkle_root, time, bits, nonce).
- Transakcje bitcoinowe i skrypty
  - Teoria:
    - Model UTXO vs Account-based. Adresy Bitcoinowe (Base58Check). Język Script (stosowy). P2PKH (Pay to Public Key Hash).
  - Zadania:
    - Ręczne tworzenie surowej transakcji (Raw Transaction) – podpisywanie wejścia kluczem prywatnym ECDSA.
    - Analiza prostego skryptu Bitcoinowego (operacje OP_DUP, OP_HASH160, OP_EQUALVERIFY).

UNIWERSYTET WSB MERITO WARSZAWA

Fundamenty i kryptografia symetryczna

Modele bezpieczeństwa i cele kryptografii

- Cele podstawowe (CIA Triad):
  - Poufność (Confidentiality):
    - Tylko uprawnieni mogą odczytać dane.
  - Integralność (Integrity):
    - Wykrywanie nieautoryzowanych zmian w danych.
  - Dostępność (Availability):
    - System działa wtedy, gdy jest potrzebny (kryptografia chroni przed DoS).
  - Dodatkowo: Niezaprzeczalność (Non-repudiation)
    - nadawca nie może wyprzeć się wysłania wiadomości

Fundamenty kryptografii

- Podstawowe pojęcia:
  - Tekst jawny (P),
  - Szyfrogram (C),
  - Klucz (K),
  - Szyfrowanie (E),
  - Deszyfrowanie (D).
- **Zasada Kerckhoffsa (XIX w.):**
  - Bezpieczeństwo systemu nie może opierać się na tajności algorytmu, lecz wyłącznie na tajności klucza.
- *Security through obscurity* (bezpieczeństwo przez ukrywanie)
  - = błąd w sztuce.

Bezpieczeństwo

- **Bezpieczeństwo obliczeniowe (Computational Security):**
  - System jest bezpieczny, jeśli najlepszy znany atak wymaga 2^{128} operacji (słońce zgaśnie zanim komputer to policzy).
  - Opiera się na trudnych problemach matematycznych (faktoryzacja, logarytm dyskretny) lub mieszaniu bitów (AES).
  - Jest warunkowe
    - może zostać złamane przez postęp w matematyce lub komputery kwantowe.

Bezpieczeństwo

- **Bezpieczeństwo informacyjne (Information-Theoretic Security):**
  - Szyfrogram nie ujawnia **żadnej** informacji o tekście jawnym (nawet przy nieskończonej mocy obliczeniowej).
  - Prawdopodobieństwo, że tekst jawny to "TAK", jest takie samo jak "NIE" po przechwyceniu szyfrogramu.
  - Jedyny przykład praktyczny: **One-Time Pad (Szyfr Vernama)**.

One-Time Pad (Szyfr Vernama)

- **Definicja:**
  - $C = M \oplus K$, gdzie M to wiadomość, K to klucz.
- **Własności XOR ($\oplus$):**
  - $x \oplus x = 0$ (samo-odwracalność).
  - $x \oplus 0 = x$ (element neutralny).
  - $A \oplus B = C \Longrightarrow C \oplus B = A$ (szyfrowanie/deszyfrowanie to ta sama operacja).
- **Warunki bezpieczeństwa absolutnego (Shannon, 1949):**
  - Klucz musi być **losowy** (jednostajny rozkład prawdopodobieństwa).
  - Klucz musi mieć długość **co najmniej równą** długości wiadomości.
  - Klucz nie może być użyty **nigdy więcej**.

Problem "Many-Time Pad" (Ponowne użycie klucza)

- Jeśli użyjemy klucza K
- dwukrotnie:
  - $C_1 = M_1 \oplus K$
  - $C_2 = M_2 \oplus K$
- Atakujący wykonuje XOR szyfrogramów:
  - $C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2 \oplus (K \oplus K) = M_1 \oplus M_2$
- **Konsekwencja:** Klucz znika z równania.
  - Otrzymujemy XOR dwóch tekstów jawnych.
- To pozwala na kryptoanalizę statystyczną (metoda "crib dragging")
  - język naturalny ma dużą redundancję, co pozwala łatwo rozdzielić M_1 i M_2.

Problem z generowaniem kluczy
Źródła losowości

- **TRNG (True Random Number Generator):**
  - Bazują na zjawiskach fizycznych (szum termiczny, rozpad promieniotwórczy, szum atmosferyczny).
  - Zaleta: Nieprzewidywalne. Wada: Wolne, wymagają sprzętu.
- **PRNG (Pseudo-RNG):**
  - Funkcja deterministyczna: $Stan_{n+1} = f(Stan_n)$.
  - Wymaga ziarna (seed). Jeśli znasz seed, znasz całą przyszłość generatora.
  - Przykłady niebezpieczne: rand() w C, random w Pythonie
    - Mersenne Twister – można odtworzyć stan po obserwacji 624 wyjść.

Problem z generowaniem kluczy
Źródła losowości

- **CSPRNG (Cryptographically Secure PRNG):**
  - PRNG, który przechodzi "Next-bit test":
    - mając ciąg bitów, nie da się przewidzieć następnego z prawdopodobieństwem $> 50\%$.
  - Odporność na kompromitację stanu (*Backtracking Resistance*)
    - (jeśli stan wycieknie, nie da się odtworzyć poprzednich kluczy.
  - Przykłady:
    - /dev/urandom (Linux),
    - Yarrow/Fortuna (MacOS/Windows),
    - ChaCha20 w trybie DRBG.

Entropia (wg Shannona)

- Miara nieuporządkowania/niepewności.
  - Hasło "password123" ma niską entropię.
  - Klucz 256-bitowy wygenerowany z rzutu monetą ma wysoką.
  - Klucz wygenerowany z czasu systemowego (time.time()) ma dramatycznie niską entropię (łatwy do zgadnięcia).

Przykład programistyczny

- Dlaczego nie używać klucza OTP dwa razy (Many-Time Pad)?
- Opis:
  - Pokażemy, że zsumowanie (XOR) dwóch szyfrogramów zaszyfrowanych tym samym kluczem usuwa klucz i zostawia XOR dwóch tekstów jawnych, co pozwala na kryptoanalizę.
- Kod do omówienia:
  - `many-time-pad.py`
- Wnioski:
  - Matematyka XOR jest bezlitosna.
  - Jeśli użyjesz klucza wielokrotnie, szyfr przestaje być bezpieczny.

Zadanie:
Łamanie generatora opartego na czasie

- Scenariusz:
  - Administrator systemu użył Pythona do wygenerowania "bezpiecznego" tokena, ale jako ziarna (seed) użył aktualnego czasu z dokładnością do sekundy.
  - Zrobił to dzisiaj, np. między godziną 12:00 a 13:00.
  - Przechwyciłeś jeden token wygenerowany przez ten system.
  - Twoim zadaniem jest znaleźć SEED (moment czasu) i przewidzieć kolejny token lub po prostu odtworzyć klucz.
- Kod startowy `server-simulation.py`
- Polecenie:
  - Napisz skrypt, który metodą Brute Force sprawdzi wszystkie możliwe wartości czasu (timestamp) z podanego zakresu.
  - Dla każdego timestampa ustaw random.seed(t), wygeneruj klucz i porównaj go z "przechwyconym".
  - Gdy znajdziesz seed, wypisz go.

UNIWERSYTET WSB MERITO WARSZAWA

Szyfry strumieniowe (Stream Ciphers)

Architektura szyfru strumieniowego

- **Idea:** Naśladowanie One-Time Pad, ale z krótszym kluczem.
- **Generator (PRNG):**
  - Rozszerza krótki klucz (np. 128 bitów) na dowolnie długi strumień klucza (Keystream).
- **Szyfrowanie:**
- $C_i = P_i \oplus S_i$
  - Gdzie S_i to bit/bajt strumienia klucza.
- **Zalety:** Bardzo szybkie, brak konieczności paddingu (szyfrujemy bit po bicie), brak propagacji błędów (błąd transmisji 1 bitu psuje tylko 1 bit po odszyfrowaniu).

Konstrukcje historyczne i współczesne

- **RC4 (Ron's Code 4):**
  - Niegdyś standard (WEP, SSL).
  - Działa na permutacji tablicy 256 bajtów.
  - *Dlaczego upadł?*
    - Pierwsze bajty strumienia nie są w pełni losowe (biases).
    - Pozwoliło to na łamanie WEP w kilka minut.
- **LFSR (Linear Feedback Shift Register):** Rejestry przesuwne.
  - Stosowane w GSM (A5/1).
  - Wada:
    - Są liniowe. Algorytm Berlekampa-Masseya pozwala odtworzyć strukturę LFSR na podstawie wyjścia o długości $2L$ (gdzie L to długość rejestru).

Konstrukcje historyczne i współczesne

- **ChaCha20 (Standard dzisiejszego internetu):**
  - Oparty na operacjach ARX (Add, Rotate, XOR) na blokach 32-bitowych.
  - Bezpieczny w implementacji
    - stały czas wykonania – odporność na ataki czasowe *Timing Attacks*.
  - Używany w TLS 1.3, VPN (WireGuard), Androidzie.

Nonce (Initialization Vector)

- Problem:
  - Klucz jest stały (Long-term key).
  - Jeśli użyjemy go dwa razy do wygenerowania strumienia, mamy problem "Many-Time Pad".
- Rozwiązanie:
  - Wejściem do generatora jest para (Klucz, Nonce).
- **Wymóg:**
  - Nonce (Number used Once) musi być unikalny dla każdego szyfrowania tym samym kluczem.
  - Nie musi być tajny, ale musi być unikalny.
- W przypadku ChaCha20:
  - 96-bitowy Nonce + 32-bitowy licznik bloków.

Malleability (kowalność) i Atak Bit-Flipping

- Szyfr jest kowalny, jeśli atakujący może zmodyfikować szyfrogram w taki sposób, że po odszyfrowaniu tekst jawny zmieni się w przewidywalny sposób.
- **Mechanizm w szyfrach strumieniowych:**
  - Mamy: $C = P \oplus K$
  - Atakujący tworzy: $C' = C \oplus \Delta$
  - Odszyfrowanie: $P' = C' \oplus K = (P \oplus K \oplus \Delta) \oplus K = P \oplus \Delta$
- **Przykład:**
  - Jeśli wiemy, że na 10-tym bajcie jest cyfra '1' (0x31), a chcemy '9' (0x39),
  - wystarczy XOR-ować 10-ty bajt szyfrogramu wartością $0x31 \oplus 0x39$
  - Odbiorca odszyfruje '9' i nie zauważy błędu (bez MAC).

Przykłady i zadanie

- Implementacja prostego LFSR
  - "szkielet" tego, jak generuje się pseudolosowość sprzętowo.
  - `simple-lfsr.py`
- **Zadanie – Atak Bit-Flipping na transakcję**
- Scenariusz: `bit-flipping.py`
  - System bankowy przesyła zaszyfrowane transakcje w formacie XML/JSON przy użyciu szyfru strumieniowego (np. AES w trybie CTR lub ChaCha20).
  - Nie używa MAC (błąd!).
  - Format wiadomości:
    `<transfer><to>Alice</to><amount>100</amount></transfer>`
- Cel:
  - przechwycenie szyfrogramu i zmiana kwoty ze "100" na "900", nie znając klucza.

UNIWERSYTET
WSB MERITO
WARSZAWA

Szyfry blokowe (Block Ciphers)
Tryby operacyjne

Definicja szyfru blokowego

- Funkcja przekształcająca blok tekstu jawnego o ustalonej długości (n bitów) w blok szyfrogramu o tej samej długości, przy użyciu klucza K
- Matematycznie: Rodzina pseudolosowych permutacji (PRP).
- **AES (Advanced Encryption Standard / Rijndael):**
  - Blok: 128 bitów (zawsze!).
  - Klucze: 128, 192, 256 bitów.
  - Struktura: SPN (Substitution-Permutation Network).
  - Składa się z rund.

Definicja Szyfru Blokowego

- **Konfuzja i Dyfuzja (Shannon):**
  - **Konfuzja:**
  - Zależność między kluczem a szyfrogramem musi być skomplikowana
    - zrealizowane przez S-Boxy – nieliniowe podstawienia.
  - **Dyfuzja:**
  - Zmiana 1 bitu tekstu jawnego powinna zmienić średnio 50% bitów szyfrogramu
    - efekt lawinowy – zrealizowane przez permutacje/mieszanie kolumn.

Padding (Dopełnianie)

- Szyfry blokowe wymagają, by dane były wielokrotnością długości bloku (16 bajtów dla AES).
- **PKCS#7:** Standard dopełniania.
  - Jeśli brakuje 1 bajtu -> dodajemy 0x01.
  - Jeśli brakuje 4 bajtów -> dodajemy 0x04, 0x04, 0x04, 0x04.
  - Jeśli wiadomość jest wielokrotnością bloku -> dodajemy cały blok 0x10 (16 bajtów)
    - aby przy deszyfrowaniu zawsze można było bezpiecznie usunąć ostatni bajt.

Tryb ECB (Electronic Codebook)

- Działanie: Dzielimy wiadomość na bloki
  - $P_1, P_2 \ldots$ i każdy szyfrujemy niezależnie: $C_i = E_K(P_i)$
- **Wady (Krytyczne!):**
  - Determinizm:
    - Identyczne bloki tekstu jawnego dają identyczne bloki szyfrogramu.
  - Ujawnia wzorce w danych.
  - Podatność na ataki **Cut-and-Paste**
    - przeklejanie bloków z jednej wiadomości do drugiej
- *Zastosowanie:* Nigdy do danych o długości > 1 bloku.

Tryb CBC (Cipher Block Chaining)

- Działanie: $C_i = E_K(P_i \oplus C_{i-1})$
- Dla pierwszego bloku używamy **IV (Initialization Vector)**:
  - $C_1 = E_K(P_1 \oplus IV)$
- **Własności:**
  - Losowe IV sprawia, że ta sama wiadomość szyfrowana dwa razy wygląda zupełnie inaczej (probabilistyczne szyfrowanie).
  - Wymaga sekwencyjnego szyfrowania (nie da się zrównoleglić).
  - Błąd transmisji w bloku C_1 niszczy P_1 całkowicie i zmienia jeden bit w P_2
- Podatny na atak **Padding Oracle** (jeśli nie jest używany z MAC).

Tryb CTR (Counter Mode)

- Zamienia szyfr blokowy w strumieniowy.
- Szyfrujemy licznik (Nonce || Counter), a wynik XOR-ujemy z wiadomością.
  - $K_{stream} = E_K(Nonce \mid\mid Counter)$
  - $C = P \oplus K_{stream}$
- Zalety:
  - Pełna równoległość (bardzo szybki), brak paddingu.
- Wady:
  - Wymaga unikalnego Nonce (jak szyfry strumieniowe), brak integralności.

Przykłady i zadanie

- ECB vs CBS
  - `tux-ecb-cbs.py`

- Zadanie – Atak "Cut-and-Paste" na ECB
- Scenariusz `cut-and-paste.py` :
  - Aplikacja webowa przechowuje sesję użytkownika w ciasteczku zaszyfrowanym AES-ECB.
  - Format danych to:
  - email=user@mail.com&uid=10&role=user
- Cel
  - spreparowanie ciasteczka, które po odszyfrowaniu da:
  - ...&role=admin

UNIWERSYTET WSB MERITO WARSZAWA

Integralność, MAC i AEAD

Uwierzytelnianie wiadomości (MAC)

- Problem:
  - Szyfrowanie (AES-CBC/CTR) chroni przed podglądaniem, ale nie przed modyfikacją.
- **MAC (Message Authentication Code):**
  - Kryptograficzna suma kontrolna.
    - $Tag = MAC(K, eyMessage)$
  - Weryfikacja: Odbiorca liczy MAC i porównuje z otrzymanym Tagiem.
- Różnica względem HASHa:
  - Hash może policzyć każdy. MAC może policzyć tylko posiadacz klucza.

HMAC (Hash-based MAC)

- Naiwna konstrukcja $H(Key \,||\, Message)$ jest niebezpieczna dla niektórych funkcji skrótu (np. SHA-2) z powodu ataku **Length Extension Attack**.
  - Atakujący widząc $H(K \,||\, M)$ może wyliczyć $H(K \,||\, M \,||\, coś)$ nie znając K
- **Konstrukcja HMAC:**
  - $H\left((K' \oplus opad) \,||\, H\big((K' \oplus ipad) \,||\, M\big)\right)$.
  - Podwójne hashowanie zapobiega przedłużaniu wiadomości.

Uwierzytelnione szyfrowanie Authenticated Encryption - AE

- Historia łączenia szyfrowania i MAC:
  - **MAC-and-Encrypt:** (SSH)
    - Szyfrujemy M, liczymy MAC z M
    - Słabe teoretycznie.
  - **MAC-then-Encrypt:** (SSL/TLS starsze)
    - Liczymy MAC z M, doklejamy do M, całość szyfrujemy.
    - Podatne na ataki Padding Oracle (bo serwer najpierw zdejmuje szyfr, sprawdza padding, a dopiero potem MAC).
  - **Encrypt-then-MAC:** (IPSec)
    - Szyfrujemy M do C. Liczymy MAC z C
    - **Jedyna matematycznie bezpieczna metoda składania.**

Uwierzytelnione Szyfrowanie Authenticated Encryption - AE

- **AEAD (Authenticated Encryption with Associated Data):**
  - Algorytmy, które robią to "w pudełku" (bezpiecznie i szybko).
  - Pozwalają też uwierzytelnić nagłówek (np. adres IP), który nie jest szyfrowany (Associated Data).
  - Standard: **AES-GCM** (Galois/Counter Mode)
    - połączenie AES-CTR z szybkim MAC-iem opartym na ciałach Galois (GHASH).

Atak Padding Oracle

- Dotyczy trybu CBC (lub starszych implementacji RSA).
- **Side-Channel Attack:**
  - Atak kanałem bocznym.
  - Atakujący nie łamie matematyki szyfru, ale wykorzystuje błędy implementacji (komunikaty błędów).
- Scenariusz:
  - Wysyłamy szyfrogram ze zmienionym ostatnim bajtem.
  - Serwer deszyfruje.
    - Jeśli padding jest błędny, zwraca błąd 500.
    - Jeśli padding dobry (ale np. zły MAC), zwraca błąd 403.
  - Ta różnica (1 bit informacji) pozwala zgadnąć zawartość bajtu metodą prób i błędów (średnio 128 zapytań na bajt).

Atak Padding Oracle

- Wnioski:
  - Zawsze używaj AEAD (AES-GCM, ChaCha20-Poly1305).
  - Jeśli musisz składać ręcznie:
    - Zawsze najpierw weryfikuj MAC, potem deszyfruj.
    - Błędy paddingu nie mogą być zwracane przed weryfikacją MAC.

Przykłady

- `hmac-sample.py`
- `padding-oracle-attack.py`

UNIWERSYTET WSB MERITO WARSZAWA

Atak Padding Oracle - szczegóły

Dygresja

Atak Padding Oracle - szczegóły

- To jest jeden z najbardziej fascynujących i najpiękniejszych logicznie ataków w kryptografii (odkryty przez Serge'a Vaudenaya w 2002 roku).
- Pozwala odszyfrować całą wiadomość **nie znając klucza**, jedynie obserwując komunikaty błędów serwera.
- **Intuicja: Włamywacz i sejf**
  - Gdy wpiszesz złą kombinację, sejf wydaje jeden z dwóch dźwięków:
    - **Głośny zgrzyt (HTTP 500):** Mechanizm się zaciął (format kodu jest błędny).
    - **Ciche "kliknięcie" (HTTP 403):** Format kodu jest poprawny, ale liczby się nie zgadzają.
- Atak Padding Oracle polega na tym, że manipulujesz zamkiem tak długo, aż usłyszysz ciche kliknięcie.
  - To "kliknięcie" to matematyczna wskazówka, która pozwala wyliczyć, co jest w środku.

Atak Padding Oracle - szczegóły

- **A. Szyfrowanie CBC (Cipher Block Chaining)**
- $P_n = D_K(C_n) \oplus C_{n-1}$
  - Gdzie:
    - P_n – Tekst jawny (Plaintext).
    - $D_K(C_n)$ –To, co wychodzi z algorytmu AES (stan pośredni, jeszcze przed XOR-em).
    - C_{n-1} – Poprzedni blok szyfrogramu (lub IV dla pierwszego bloku).
- **Kluczowa obserwacja:**
  - Atakujący widzi i może zmieniać C_{n-1}
  - Jeśli zmienisz jeden bajt w C_{n-1} ,zmienisz odpowiadający mu bajt w P_n po odszyfrowaniu!

Atak Padding Oracle - szczegóły

- **B. Padding PKCS#7**
  Wiadomość musi być wielokrotnością 16 bajtów.
  Końcówka **musi** wyglądać tak:
  - ... 01 (jeśli brakowało 1 bajtu)
  - ... 02 02 (jeśli brakowało 2 bajtów)
  - ... 03 03 03 (jeśli brakowało 3 bajtów)
  - ...itd.
- Jeśli po odszyfrowaniu końcówka to np. ... 05 02 01, serwer wie, że to bzdura (Padding Error).

Atak Padding Oracle - szczegóły

- **Błędy 500 i 403 -** Wyrocznia
- Scenariusz:
  - Atakujący wysyła zmodyfikowany szyfrogram do serwera.
  - Serwer wykonuje kroki:
    - Deszyfruje AESem.
    - Sprawdza padding (czy na końcu jest 01, 02 02 itd.).
    - Jeśli padding jest OK -> sprawdza treść (np. czy to poprawny JSON, czy hasło pasuje).
- **Reakcja serwera (Wyrocznia):**

Atak Padding Oracle - szczegóły

- **HTTP 500 (Internal Server Error):**
  - Serwer próbował zdjąć padding (funkcja unpad()).
  - Algorytm zobaczył na końcu np. wartość 0x3F (63), ale poprzednie bajty nie pasowały do wzorca paddingu.
  - Wyrzucił wyjątek PaddingException.
- **Wniosek dla hakera:**
  - "Moja manipulacja sprawiła, że końcówka odszyfrowanego tekstu jest losowym śmieciem".

- **HTTP 403 (Forbidden) / 200 (OK) / Błąd logowania:**
  - Funkcja unpad() zadziałała! Padding był poprawny (np. 01).
  - Serwer przeszedł do przetwarzania danych, ale np. HMAC się nie zgadzał albo struktura danych była zła.
- **Wniosek dla hakera:**
  - "BINGO! Moja manipulacja sprawiła, że ostatni bajt odszyfrowanego tekstu to dokładnie 0x01 (lub inna poprawna końcówka)".

Atak Padding Oracle - szczegóły

- **Matematyka ataku (Krok po kroku)**
- Chcemy odszyfrować ostatni bajt bloku C_2 nazwijmy go $P_2[15]$
- Mamy szyfrogram: C_1 i C_2
- Wiemy, że: $P_2[15] = D_K(C_2)[15] \oplus C_1[15]$
- Nie znamy $D_K(C_2)$ ani klucza K
  - Ale znamy C_1 (bo go przechwyciliśmy).

Atak Padding Oracle - szczegóły

- **Krok 1: Manipulacja**
- Tworzymy fałszywy blok
  - C_1' (Fake IV), który jest kopią C_1 ,ale zmieniamy ostatni bajt.
  - Próbujemy wszystkich wartości od 0 do 255.
- Wysyłamy do serwera parę: (C_1', C_2)

Atak Padding Oracle - szczegóły

- **Krok 2: Szukanie "Kliknięcia" (Błąd 403)**
- Serwer deszyfruje C_2. Otrzymuje jakiś "Stan Pośredni" (IS).
- Potem robi XOR z naszym C_1'
  - $P_2' = IS \oplus C_1'$
- Dla 255 prób serwer zwróci **HTTP 500** (błąd paddingu).
- Ale dla jednej wartości X (którą wstawiliśmy na koniec C_1') wynik po odszyfrowaniu zakończy się na 0x01.
  - Serwer uzna to za poprawny padding i zwróci **HTTP 403**.

Atak Padding Oracle - szczegóły

- **Krok 3: Obliczenia**
- W momencie, gdy serwer zwrócił 403, wiemy na 100%, że ostatni bajt tekstu jawnego (dla serwera) wynosił 0x01.
- Zapiszmy równanie dla tego momentu:
  - $0x01 = IS[15] \oplus X$
    - Gdzie X to wartość, którą wstrzyknęliśmy i która zadziałała.
- Dzięki własnościom XOR możemy obliczyć sekretny Stan Pośredni (IS):
  - $IS[15] = 0x01 \oplus X$

Atak Padding Oracle - szczegóły

- **Krok 4: Odszyfrowanie oryginału**
- Skoro znamy $IS[15]$, możemy obliczyć, jaki był **oryginalny** tekst jawny, używając **oryginalnego** C_1 (tego, który przechwyciliśmy na początku):
  - $P_{orig}[15] = IS[15] \oplus C_1[15]$
- I mamy to!
  - Odszyfrowaliśmy jeden bajt bez klucza.

Atak Padding Oracle - szczegóły

- **Jak idziemy dalej? (kolejne bajty)**
- Chcemy odszyfrować przedostatni bajt ($P[14]$)
- Musimy zmusić serwer, by uznał padding za poprawny.
  - Tym razem poprawny padding to 0x02, 0x02.
  - Znamy już $IS[15]$
  - Ustawiamy ostatni bajt naszego fałszywego C_1' tak, aby po odszyfrowaniu dał 0x02.
    - $C_1'[15] = IS[15] \oplus 0x02$
- Teraz pętlą zmieniamy przedostatni bajt ($C_1'[14]$) szukając takiej wartości, która sprawi, że serwer nie zwróci błędu 500. Gdy trafimy, oznacza to, że końcówka to 02 02.
  - Wyliczamy $IS[14] = C_1'[14] \oplus 0x02$.
- Powtarzamy to aż do odszyfrowania całego bloku.

Do zobaczenia ☺